

LINUX NETWORKING & TROUBLESHOOTING COMMAND REFERENCE FOR NOC ANALYSTS

A Practical, Screenshot Based Portfolio Project

Prepared by:

Hassan Khan

Aspiring NOC / Network Operations Analyst

Environment: Kali Linux (VirtualBox / VMware)

Date: July 2026

Project Objective

This project is a practical command reference built for Network Operations Center (NOC) Analysts. It documents 30 essential Linux commands used daily to check network configuration, troubleshoot connectivity, analyze routing, resolve DNS issues, check open ports, monitor active connections, investigate running processes, verify service health, and review system logs.

Each command entry includes its purpose, syntax, a real-world NOC use case, an example, expected output, a checklist of what to verify, and a terminal screenshot demonstrating the command running on a live Kali Linux system. The goal is to demonstrate hands-on Linux networking and troubleshooting competency for a NOC Analyst / Network Support role.

Tools & Environment

- Operating System: Kali Linux (Desktop/Server) running in a Virtual Machine (VMware)
- Terminal: GNOME Terminal / Bash shell
- Networking utilities: iproute2, iputils-ping, traceroute, tracepath, mtr, curl, bind9-dnsutils, systemd-resolved, iproute2, net-tools, lsof, netcat, telnet, ethtool, tcpdump
- System utilities: systemd (systemctl, journalctl), procs (ps, top), coreutils (df)

Note: Most of these utilities are installed by default on Ubuntu. Any missing tool can be installed with: `sudo apt install <package-name>`

Section 1 Network Configuration

These commands help a NOC Analyst confirm that a server's network interfaces, IP addresses, and routing configuration match what is expected before deeper troubleshooting begins.

1. ip addr

Purpose:

Displays all network interfaces on the system along with their assigned IP addresses, subnet masks, and interface state.

Syntax:

```
ip addr
ip addr show
ip addr show eth0
```

NOC Use Case:

A NOC Analyst uses this command first when a server reports connectivity issues, to confirm the interface is UP and has the expected IP address and subnet mask.

Example:

```
ip addr
```

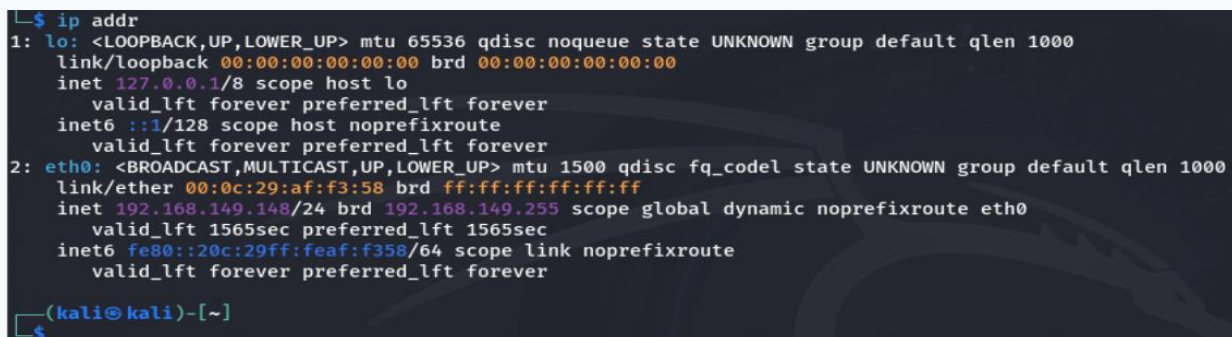
Expected Output:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
   inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
   link/ether 08:00:27:4a:3c:9d brd ff:ff:ff:ff:ff:ff
   inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
```

What to Check:

- Is the interface state UP (not DOWN)?
- Is the expected IP address assigned?
- Is the subnet mask (/24, /16, etc.) correct?
- Is the loopback interface (lo) also up?

Screenshot:



```

$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN group default qlen 1000
   link/ether 00:0c:29:af:f3:58 brd ff:ff:ff:ff:ff:ff
   inet 192.168.149.148/24 brd 192.168.149.255 scope global dynamic noprefixroute eth0
       valid_lft 1565sec preferred_lft 1565sec
   inet6 fe80::20c:29ff:feaf:f358/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
(kali@kali)~$

```

**SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: `ip addr`

2. `ip link`

Purpose:

Shows the state of network interfaces at Layer 2, including MAC address and link status, without IP address details.

Syntax:

```
ip link
ip link show eth0
```

NOC Use Case:

A NOC Analyst uses this to isolate a problem to Layer 2 — confirming the physical/virtual link is up before checking IP configuration.

Example:

```
ip link
```

Expected Output:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP
   link/ether 08:00:27:4a:3c:9d brd ff:ff:ff:ff:ff:ff
```

What to Check:

- Is the interface state UP?
- Is LOWER_UP present (physical/virtual link detected)?
- Does the MAC address match the expected NIC?

**Screenshot:**

```
(kali㉿kali)-[~]
$ ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UNKNOWN mode DEFAULT group default qlen 1000
   link/ether 00:0c:29:af:f3:58 brd ff:ff:ff:ff:ff:ff

(kali㉿kali)-[~]
$
```

**SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: `ip link`

3. ip route

Purpose:

Displays the system's routing table, showing which gateway and interface traffic uses to reach different networks.

Syntax:

```
ip route
ip route show
```

NOC Use Case:

A NOC Analyst checks this when a server can reach some networks but not others, to confirm a default gateway exists and routes point out the correct interface.

Example:

```
ip route
```

Expected Output:

```
default via 192.168.1.1 dev eth0 proto static
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100
```

What to Check:

- Is there a default route (default via ...)?
- Does the gateway IP match the expected router?
- Is the route bound to the correct interface (dev)?

** Screenshot:**

```
(kali㉿kali)~[~]
$ ip rou
default via 192.168.149.2 dev eth0 proto dhcp src 192.168.149.148 metric 100
192.168.149.0/24 dev eth0 proto kernel scope link src 192.168.149.148 metric 100

(kali㉿kali)~[~]
$ s
```

** SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: ip route

4. hostname

Purpose:

Displays the configured hostname of the Linux system.

Syntax:

```
hostname
```

NOC Use Case:

A NOC Analyst confirms they are working on the correct server, especially during remote sessions across many systems.

Example:

```
hostname
```

Expected Output:

```
noc-server01
```

What to Check:

- Does the hostname match the expected server naming convention?
- Is this the correct target system for the task at hand?

** Screenshot:**

```
(kali@kali)-[~]  
$ hostname  
kali
```

** SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: hostname

5. hostname -I

Purpose:

Displays all IP addresses assigned to the system in a single line, useful for quick scripting or verification.

Syntax:

```
hostname -I
```

NOC Use Case:

A NOC Analyst uses this for a fast IP check without parsing the full ip addr output, especially in scripts or automated health checks.

Example:

```
hostname -I
```

Expected Output:

```
192.168.1.100
```

What to Check:

- Does the output list the expected IP address(es)?
- Are there unexpected extra IPs (e.g., from a second NIC or VPN)?

 Screenshot:

```
(kali㉿kali)-[~]  
└─$ hostname -I  
192.168.149.148  
  
(kali㉿kali)-[~]  
└─$
```

 SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: `hostname -I`

Section 2 Connectivity Troubleshooting

These commands are used to test reachability, measure latency, and identify where along a network path a connection is failing.

6. ping

Purpose:

Tests basic reachability between the local host and a remote host using ICMP echo requests, and measures round-trip latency.

Syntax:

```
ping -c 4 8.8.8.8
ping -c 4 google.com
```

NOC Use Case:

A NOC Analyst uses ping as the first troubleshooting step to confirm whether a remote host is reachable at all, and to check for packet loss or high latency.

Example:

```
ping -c 4 8.8.8.8
```

Expected Output:

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=115 time=12.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=115 time=11.9 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=115 time=12.7 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=115 time=12.1 ms

--- 8.8.8.8 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 11.9/12.275/12.7/0.303 ms
```

What to Check:

- Is packet loss 0% or high?
- Is round-trip time (RTT) within normal range for this link?
- Does the request time out completely (host unreachable)?



Screenshot:


```
(kali@kali)-[~]  
$ ping -c 4 8.8.8.8  
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=128 time=59.5 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=128 time=66.7 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=128 time=61.5 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=128 time=64.4 ms  
  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3012ms  
rtt min/avg/max/mdev = 59.484/63.028/66.653/2.733 ms  
  
(kali@kali)-[~]
```

 SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: ping

7. traceroute

Purpose:

Shows the path (each router hop) that packets take to reach a destination, along with the latency at each hop.

Syntax:

```
traceroute 8.8.8.8  
traceroute google.com
```

NOC Use Case:

A NOC Analyst uses this to identify exactly which hop along the path is dropping packets or introducing high latency, narrowing down whether the issue is local, ISP-side, or on the destination network.

Example:

```
traceroute 8.8.8.8
```

Expected Output:

```
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets  
 1 192.168.1.1  1.021 ms  0.987 ms  0.954 ms  
 2 10.10.0.1  5.432 ms  5.401 ms  5.388 ms  
 3 * * *  
 4 72.14.215.85 11.203 ms 11.187 ms 11.150 ms  
 5 8.8.8.8 12.401 ms 12.377 ms 12.350 ms
```

What to Check:

- Where does latency suddenly increase?
- Are there hops showing * * * (no response)?
- Does the trace complete and reach the destination?

 Screenshot:

```
(kali㉿kali)-[~]  
$ traceroute 8.8.8.8  
traceroute to 8.8.8.8 (8.8.8.8), 30 hops max, 60 byte packets  
 1  _gateway (192.168.149.2)  0.276 ms  0.361 ms  0.148 ms  
 2  * * *  
 3  * * *  
 4  * * *  
 5  * * *  
 6  * * *  
 7  * * *  
 8  * * *  
 9  * * *  
10  * * *  
11  * * *  
12  * * *  
13  * * *  
14  * * *  
15  * * *  
16  * * *  
17  * * *  
18  * * *  
19  * * *  
20  * * *  
21  * * *  
22  * * *  
23  * * *  
24  * * *  
25  * * *  
26  * * *  
27  * * *  
28  * * *  
29  * * *  
30  * * *
```

 SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: traceroute

8. tracepath

Purpose:

A lighter alternative to traceroute that does not require root privileges and also reports the path MTU.

Syntax:

```
tracert 8.8.8.8
```

NOC Use Case:

A NOC Analyst uses this on systems where traceroute is unavailable or root access is restricted, to quickly trace the path and check for MTU-related issues.

Example:

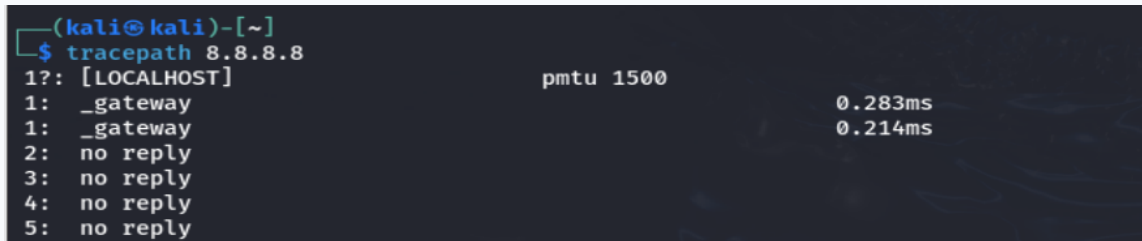
```
tracert 8.8.8.8
```

Expected Output:

```
1?: [LOCALHOST]  pmtu 1500
1:  192.168.1.1  1.045ms
2:  10.10.0.1   5.512ms
3:  no reply
4:  72.14.215.85 11.301ms asymm 5
5:  8.8.8.8  12.410ms reached
    Resume: pmtu 1500 hops 5 back 5
```

What to Check:

- Is the path MTU as expected (commonly 1500)?
- Are there 'no reply' hops indicating filtering or drops?
- Did the trace report 'reached' at the final hop?

🖼️ Screenshot:


```
(kali㉿kali)-[~]
$ tracert 8.8.8.8
1?: [LOCALHOST]           pmtu 1500
1:  _gateway               0.283ms
1:  _gateway               0.214ms
2:  no reply
3:  no reply
4:  no reply
5:  no reply
```

🖼️ SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: tracert

9. mtr

Purpose:

Combines ping and traceroute into a continuously updating view, showing packet loss and latency per hop over time.

Syntax:

```
mtr 8.8.8.8
mtr -rw 8.8.8.8
```

NOC Use Case:

A NOC Analyst uses mtr for ongoing intermittent connectivity issues, since it keeps sampling each hop over time rather than a single snapshot, making intermittent loss easier to spot.

Example:

```
mtr -rw -c 10 8.8.8.8
```

Expected Output:

```
Start: 2026-07-26T10:00:00+0000
HOST: noc-server01
```

	Loss%	Snt	Last	Avg	Best	Wrst	StDev
1. 192.168.1.1	0.0%	10	1.0	1.1	0.9	1.4	0.1
2. 10.10.0.1	0.0%	10	5.4	5.5	5.3	5.9	0.2
3. ???	100.0%	10	0.0	0.0	0.0	0.0	0.0
4. 72.14.215.85	0.0%	10	11.2	11.3	11.1	11.6	0.1
5. 8.8.8.8	0.0%	10	12.4	12.4	12.3	12.6	0.1

What to Check:

- Which hop(s) show sustained packet loss %?
- Is loss at a middle hop but 0% at the final hop (often just ICMP deprioritization, not a real problem)?
- Is latency (Avg/Wrst) stable or spiking?

🖥️ Screenshot:

```
(kali@kali)-[~]
$ mtr -rw -c 10 8.8.8.8
Start: 2026-07-26T23:21:01-0500
HOST: kali
```

	Loss%	Snt	Last	Avg	Best	Wrst	StDev
1. -- _gateway	0.0%	10	0.5	0.5	0.5	1.0	0.2
2. -- 10.49.36.26	0.0%	10	4.8	8.8	3.2	20.0	5.1
3. -- ???	100.0	10	0.0	0.0	0.0	0.0	0.0
4. -- 10.180.168.146	0.0%	10	75.2	47.5	22.3	101.2	26.9
5. -- 10.81.235.46	0.0%	10	27.2	41.7	19.6	69.8	18.1
6. -- 10.80.238.134	0.0%	10	32.8	42.0	23.6	86.8	17.8
7. -- 119.156.25.49	0.0%	10	26.5	37.2	23.3	69.5	17.3
8. -- 10.253.8.44	0.0%	10	58.9	34.2	17.9	58.9	13.2
9. -- 10.253.4.36	0.0%	10	105.0	55.6	33.8	105.0	22.5
10. -- 10.253.4.26	0.0%	10	51.3	53.9	33.1	98.0	19.0
11. -- 74.125.51.180	0.0%	10	72.2	66.2	55.0	79.3	8.5
12. -- 209.85.241.201	0.0%	10	72.9	90.5	60.3	232.7	51.1
13. -- 172.253.66.41	0.0%	10	52.9	75.3	51.6	188.5	40.7
14. -- dns.google	0.0%	10	54.3	71.7	53.1	116.7	20.8

```
(kali@kali)-[~]
$
```

🖥️ SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: mtr

10. curl

Purpose:

Sends an HTTP/HTTPS (or other protocol) request to a server and returns the response, headers, and status code.

Syntax:

```
curl -I https://example.com
curl -v https://example.com
```

NOC Use Case:

A NOC Analyst uses curl to confirm whether a web service is responding correctly at the application layer, even when ping and traceroute show network-level reachability is fine.

Example:

```
curl -I https://example.com
```

Expected Output:

```
HTTP/2 200
content-type: text/html; charset=UTF-8
date: Sun, 26 Jul 2026 10:00:00 GMT
server: ECS (nyb/1D2A)
content-length: 1256
```

What to Check:

- Is the HTTP status code 200 (success) or an error (4xx/5xx)?
- Is the response time acceptable?
- Are expected headers (server, content-type) present?



Screenshot:

```
(kali@kali)-[~]
$ curl -I https://google.com
HTTP/2 301
location: https://www.google.com/
content-type: text/html; charset=UTF-8
content-security-policy-report-only: object-src 'none';base-uri 'self';script-src 'nonce-IAAr1E2Yja4koUtQMz13gQ' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other-hp
date: Mon, 27 Jul 2026 04:22:20 GMT
expires: Wed, 26 Aug 2026 04:22:20 GMT
cache-control: public, max-age=2592000
server: gws
content-length: 220
x-xss-protection: 0
x-frame-options: SAMEORIGIN
alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
(kali@kali)-[~]
```



SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: curl

Section 3 DNS Troubleshooting

These commands help a NOC Analyst determine whether a connectivity problem is actually a DNS resolution problem, which is one of the most common root causes reported to a NOC.

11. nslookup

Purpose:

Queries DNS servers to resolve a hostname to an IP address (or vice versa).

Syntax:

```
nslookup google.com
nslookup google.com 8.8.8.8
```

NOC Use Case:

A NOC Analyst uses nslookup to quickly confirm whether a domain resolves correctly and which DNS server answered the query.

Example:

```
nslookup google.com
```

Expected Output:

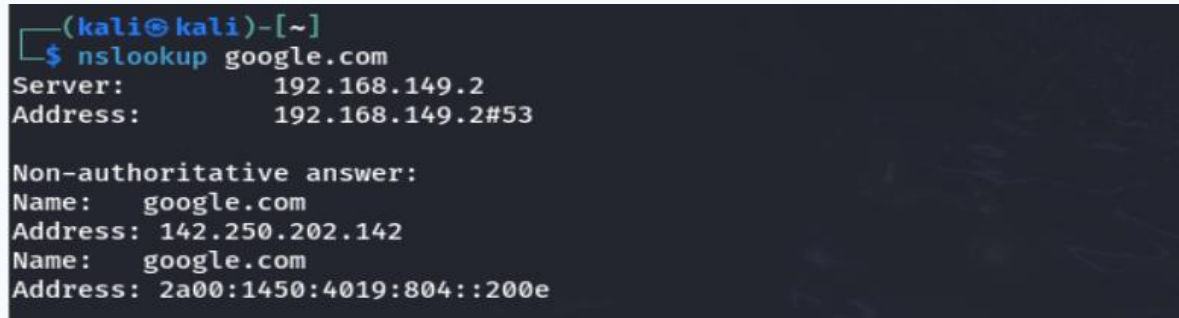
```
Server: 127.0.0.53
Address: 127.0.0.53#53

Non-authoritative answer:
Name: google.com
Address: 142.250.72.14
```

What to Check:

- Did the domain resolve to an IP at all?
- Does the resolved IP match what is expected?
- Which DNS server answered the query?

Screenshot:



```
(kali㉿kali)-[~]
$ nslookup google.com
Server:      192.168.149.2
Address:     192.168.149.2#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.202.142
Name:   google.com
Address: 2a00:1450:4019:804::200e
```

SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: nslookup

12. dig

Purpose:

Performs detailed DNS queries and returns full response records, including TTL, query time, and record type.

Syntax:

```
dig google.com
dig google.com MX
dig @8.8.8.8 google.com
```

NOC Use Case:

A NOC Analyst uses dig for deeper DNS investigation than nslookup provides, especially to check TTLs, specific record types (A, MX, CNAME), and to test against a specific DNS server.

Example:

```
dig google.com
```

Expected Output:

```
; <<>> DiG 9.18.1 <<>> google.com
;; ANSWER SECTION:
google.com.    205  IN  A   142.250.72.14

;; Query time: 14 msec
;; SERVER: 127.0.0.53#53(127.0.0.53)
```

What to Check:

- Does the ANSWER SECTION contain the expected record?
- Is the TTL reasonable, or unusually low/high?
- Is query time high, suggesting a slow DNS server?



Screenshot:

```
(kali㉿kali)-[~]
$ dig google.com

; <>> DiG 9.20.0-Debian <>> google.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 5634
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 0

;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                  5       IN      A      142.250.202.142

;; Query time: 11 msec
;; SERVER: 192.168.149.2#53(192.168.149.2) (UDP)
;; WHEN: Sun Jul 26 23:25:11 CDT 2026
;; MSG SIZE rcvd: 44
```

 **SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: dig

13. resolvectl

Purpose:

Shows the current DNS resolver configuration used by systemd-resolved, including per-interface DNS servers.

Syntax:

```
resolvectl status
resolvectl query google.com
```

NOC Use Case:

A NOC Analyst uses this on modern Ubuntu systems to see exactly which DNS servers are active per interface, which is useful when multiple network interfaces (e.g., VPN + LAN) are in use.

Example:

```
resolvectl status
```

Expected Output:

```
Global
    Protocols: -LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported

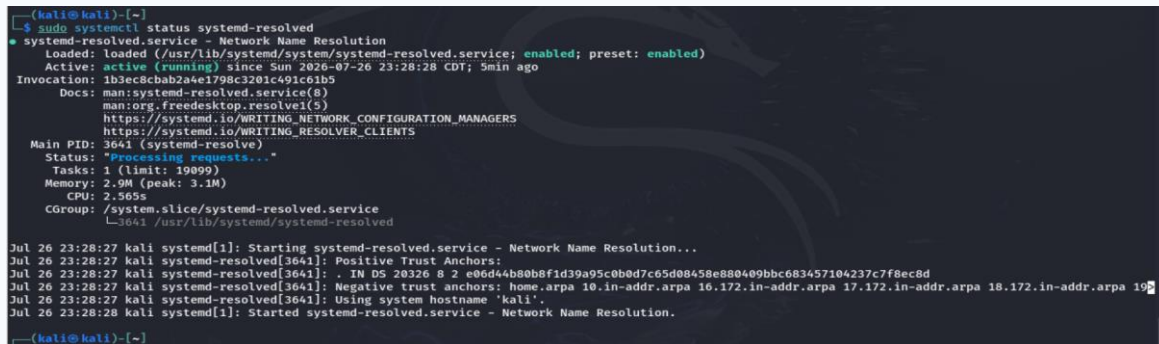
Link 2 (eth0)
    Current Scopes: DNS
    DefaultRoute setting: yes
    Current DNS Server: 192.168.1.1
```


DNS Servers: 192.168.1.1 8.8.8.8

What to Check:

- Which DNS server is currently active for the interface?
- Are the expected DNS servers listed?
- Is DNSSEC or DNSOverTLS unexpectedly disabled/enabled?

Screenshot:



```
(kali@kali)~$ sudo systemctl status systemd-resolved
● systemd-resolved.service - Network Name Resolution
   Loaded: loaded (/usr/lib/systemd/system/systemd-resolved.service; enabled; preset: enabled)
   Active: active (running) since Sun 2026-07-26 23:28:28 CDT; 5min ago
     Invocation: 1b3ec8cbab2a4e1798c3201c491c61b5
       Docs: man:systemd-resolved.service(8)
             man:org.freedesktop.resolve1(5)
             https://systemd.io/Writing_Network_Configuration_Managers
             https://systemd.io/Writing_Resolver_Clients
    Main PID: 3641 (systemd-resolve)
      Status: "Processing requests..."
        Tasks: 1 (limit: 19099)
       Memory: 2.9M (peak: 3.1M)
          CPU: 2.565s
      CGroup: /system.slice/systemd-resolved.service
              └─3641 /usr/lib/systemd/systemd-resolved

Jul 26 23:28:27 kali systemd[1]: Starting systemd-resolved.service - Network Name Resolution...
Jul 26 23:28:27 kali systemd-resolved[3641]: Positive Trust Anchors:
Jul 26 23:28:27 kali systemd-resolved[3641]: . IN DS 20326 8 2 e06d44b80b8f1d39a95c0b0d7c65d08458e880409bbc683457104237c7f8ec8d
Jul 26 23:28:27 kali systemd-resolved[3641]: Negative trust anchors: home.arpa 10.in-addr.arpa 16.172.in-addr.arpa 17.172.in-addr.arpa 18.172.in-addr.arpa 19
Jul 26 23:28:27 kali systemd-resolved[3641]: Using system hostname 'kali'.
Jul 26 23:28:28 kali systemd[1]: Started systemd-resolved.service - Network Name Resolution.

(kali@kali)~$
```

SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: resolvectl

14. cat /etc/resolv.conf

Purpose:

Displays the static DNS resolver configuration file, listing the nameservers the system is configured to use.

Syntax:

```
cat /etc/resolv.conf
```

NOC Use Case:

A NOC Analyst checks this file when DNS resolution fails, to confirm the correct nameserver IPs are configured and haven't been overwritten or left empty.

Example:

```
cat /etc/resolv.conf
```

Expected Output:

```
nameserver 127.0.0.53
options edns0 trust-ad
search localdomain
```

What to Check:

- Are nameserver entries present (file not empty)?

- Do the nameserver IPs match the expected DNS servers?
- Is the search domain correct for the environment?

Screenshot:

```
(kali@kali)-[~]
$ cat /etc/resolv.conf
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search .
```

SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: cat /etc/resolv.conf

15. getent hosts

Purpose:

Resolves a hostname using the system's full name resolution chain (DNS, /etc/hosts, NSS), the same way applications resolve names.

Syntax:

```
getent hosts google.com
```

NOC Use Case:

A NOC Analyst uses this to confirm what an application will actually resolve a hostname to, since it reflects /etc/hosts and NSS ordering, not just raw DNS — useful for catching stale /etc/hosts entries.

Example:

```
getent hosts google.com
```

Expected Output:

```
142.250.72.14 google.com
```

What to Check:

- Does the result match what dig/nslookup returned?
- If it differs, is a stale /etc/hosts entry overriding DNS?

** Screenshot:**

```
(kali@kali)-[~]  
$ getent hosts google.com
```

** SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: getent hosts

Section 4 Ports & Connections

These commands let a NOC Analyst confirm which ports are open, which processes own them, and what active connections exist on the system.

16. ss

Purpose:

Displays active sockets, listening ports, and established connections — a modern, faster replacement for netstat.

Syntax:

```
ss -tuln
ss -tulnp
```

NOC Use Case:

A NOC Analyst uses ss to quickly confirm whether a service is listening on the expected port and interface, and to see all current TCP/UDP connections.

Example:

```
ss -tuln
```

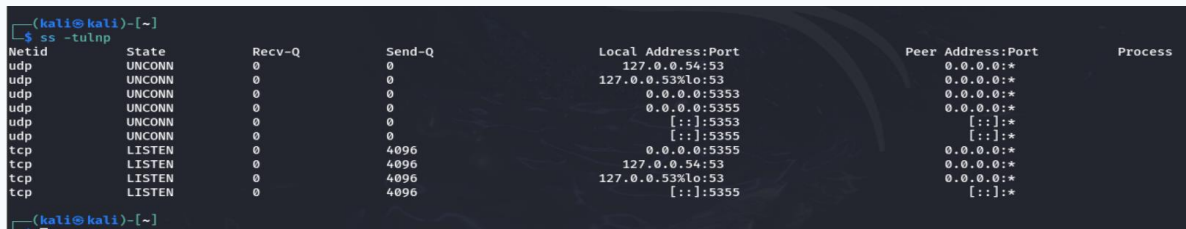
Expected Output:

Netid	State	Local Address:Port	Peer Address:Port
tcp	LISTEN	0.0.0.0:22	0.0.0.0:*
tcp	LISTEN	0.0.0.0:80	0.0.0.0:*
tcp	ESTAB	192.168.1.100:22	192.168.1.55:52344

What to Check:

- Is the expected port in LISTEN state?
- Is it bound to the correct address (0.0.0.0 vs 127.0.0.1)?
- Are there unexpected established connections?

Screenshot:



```
(kali@kali)~$ ss -tulnp
Netid  State      Recv-Q    Send-Q    Local Address:Port    Peer Address:Port    Process
udp    UNCONN    0          0          127.0.0.54:53         0.0.0.0:*             nftables
udp    UNCONN    0          0          127.0.0.53%lo:53      0.0.0.0:*             nftables
udp    UNCONN    0          0          0.0.0.0:5353         0.0.0.0:*             nftables
udp    UNCONN    0          0          0.0.0.0:5355         0.0.0.0:*             nftables
udp    UNCONN    0          0          [::]:5353            [::]:*                 nftables
udp    UNCONN    0          0          [::]:5355            [::]:*                 nftables
tcp    LISTEN    0          4096      0.0.0.0:5355         0.0.0.0:*             nftables
tcp    LISTEN    0          4096      127.0.0.54:53        0.0.0.0:*             nftables
tcp    LISTEN    0          4096      127.0.0.53%lo:53     0.0.0.0:*             nftables
tcp    LISTEN    0          4096      [::]:5355           [::]:*                 nftables
```

 **SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: ss

17. netstat

Purpose:

Displays network connections, routing tables, and listening ports (legacy tool, still widely used and recognized).

Syntax:

```
netstat -tuln
netstat -r
```

NOC Use Case:

A NOC Analyst uses netstat in environments or scripts where it is still the standard, to check listening ports and active connections in the same way ss does.

Example:

```
netstat -tuln
```

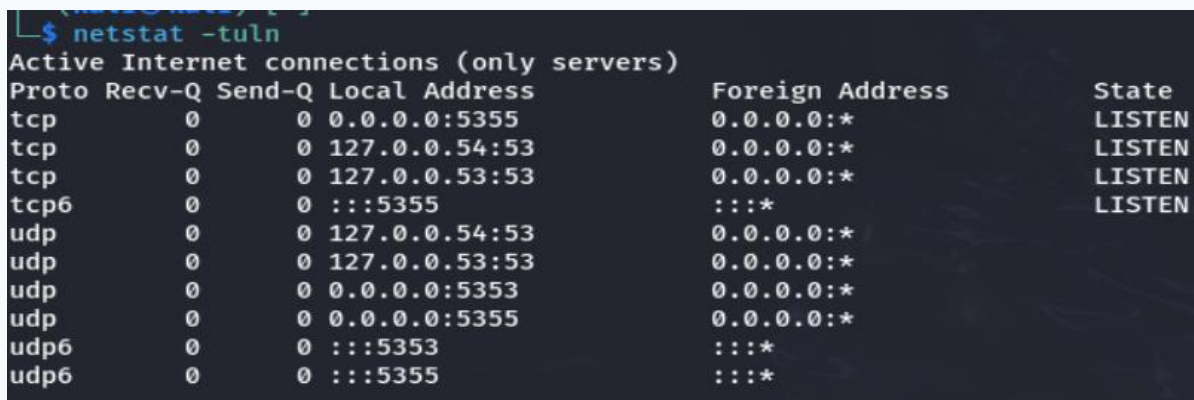
Expected Output:

Proto	Local Address	Foreign Address	State
tcp	0.0.0.0:22	0.0.0.0:*	LISTEN
tcp	0.0.0.0:80	0.0.0.0:*	LISTEN

What to Check:

- Is the service port listed as LISTEN?
- Is the port bound to the correct interface?

Screenshot:



```

$ netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:5355             0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.54:53            0.0.0.0:*               LISTEN
tcp        0      0 127.0.0.53:53            0.0.0.0:*               LISTEN
tcp6       0      0 :::5355                  :::*                     LISTEN
udp        0      0 127.0.0.54:53            0.0.0.0:*
udp        0      0 127.0.0.53:53            0.0.0.0:*
udp        0      0 0.0.0.0:5353             0.0.0.0:*
udp        0      0 0.0.0.0:5355             0.0.0.0:*
udp6       0      0 :::5353                  :::*
udp6       0      0 :::5355                  :::*

```



SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: netstat

18. lsof

Purpose:

Lists open files, and with the `-i` flag, shows which processes own which network sockets/ports.

Syntax:

```
lsof -i :80
lsof -i -P -n
```

NOC Use Case:

A NOC Analyst uses `lsof` when a port is unexpectedly in use or a service won't start due to 'address already in use', to identify exactly which process (PID) holds the port.

Example:

```
lsof -i :80
```

Expected Output:

COMMAND	PID	USER	FD	TYPE	NODE	NAME
nginx	1042	root	6u	IPv4	TCP	*:80 (LISTEN)
nginx	1043	www-data	6u	IPv4	TCP	*:80 (LISTEN)

What to Check:

- Which process/PID owns the port?
- Is this the expected service, or an unexpected/rogue process?

Screenshot:



```
(kali㉿kali)-[~]
$ lsof -i -P -n

(kali㉿kali)-[~]
$ lsof -i :80
```

 **SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: `lsof`

19. nc

Purpose:

Netcat — tests whether a specific TCP or UDP port is open and reachable on a remote host.

Syntax:

```
nc -zv 192.168.1.10 443
nc -zv 192.168.1.10 20-25
```

NOC Use Case:

A NOC Analyst uses `nc` to confirm firewall/port-level reachability between two hosts without needing the full application (e.g., testing port 443 is open before troubleshooting an HTTPS issue).

Example:

```
nc -zv 192.168.1.10 443
```

Expected Output:

```
Connection to 192.168.1.10 443 port [tcp/https] succeeded!
```

What to Check:

- Did the connection succeed or time out/refuse?
- Is this consistent with expected firewall rules?



Screenshot:

```
(kali㉿kali)-[~]
$ nc -zv 192.168.1.10 443
192.168.1.10: inverse host lookup failed: Host name lookup failure : Resource temporarily unavailable
(UNKNOWN) [192.168.1.10] 443 (https) : Connection refused
```



SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: nc

20. telnet

Purpose:

Opens a raw TCP connection to a host and port, historically used for remote login but now mainly used for port testing.

Syntax:

```
telnet 192.168.1.10 80
```

NOC Use Case:

A NOC Analyst uses `telnet` as an alternative to `nc` to test TCP port connectivity and, when connected, manually send protocol commands (e.g., HTTP GET) to inspect raw responses.

Example:

```
telnet 192.168.1.10 80
```

Expected Output:

```
Trying 192.168.1.10...
Connected to 192.168.1.10.
Escape character is '^['.
```

What to Check:

- Did it say 'Connected to...' (port open) or 'Connection refused' / hang (port closed/filtered)?
-

Section 5 Network Information & Traffic

These commands give a NOC Analyst deeper visibility into the local network segment, hardware-level interface stats, and live packet capture for deep-dive troubleshooting.

21. arp

Purpose:

Displays the ARP table, mapping IP addresses to MAC addresses on the local network segment.

Syntax:

```
arp -a  
arp -n
```

NOC Use Case:

A NOC Analyst uses arp to confirm that a local device is being seen correctly on the LAN, and to detect IP conflicts (two MACs claiming the same IP).

Example:

```
arp -a
```

Expected Output:

```
? (192.168.1.1) at 00:11:22:33:44:55 [ether] on eth0  
? (192.168.1.55) at aa:bb:cc:dd:ee:ff [ether] on eth0
```

What to Check:

- Is the expected device's MAC address present?
- Is any IP mapped to more than one MAC (possible conflict)?



Screenshot:

```
(kali㉿kali)-[~]  
$ arp -a  
_gateway (192.168.149.2) at 00:50:56:f7:16:69 [ether] on eth0
```



SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: arp

22. ip neigh

Purpose:

The modern equivalent of arp -a, showing the kernel's neighbor table (IP-to-MAC mappings) with reachability state.

Syntax:

```
ip neigh
ip neigh show
```

NOC Use Case:

A NOC Analyst uses this to check not just the MAC mapping but also the neighbor's reachability state (REACHABLE, STALE, FAILED), which helps diagnose local-segment issues.

Example:

```
ip neigh
```

Expected Output:

```
192.168.1.1 dev eth0 lladdr 00:11:22:33:44:55 REACHABLE
192.168.1.55 dev eth0 lladdr aa:bb:cc:dd:ee:ff STALE
```

What to Check:

- Are entries REACHABLE, or FAILED/STALE?
- Does the MAC address match the expected device?

**Screenshot:**

```
(kali@kali)-[~]
$ ip neigh show
192.168.149.2 dev eth0 lladdr 00:50:56:f7:16:69 STALE
```

**SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: ip neigh

23. ethtool

Purpose:

Displays and configures low-level NIC settings such as link speed, duplex mode, and auto-negotiation status.

Syntax:

```
ethtool eth0
```

NOC Use Case:

A NOC Analyst uses ethtool to rule out physical-layer issues such as a NIC negotiating the wrong speed/duplex, which can cause slow throughput even when routing and DNS are fine.

Example:

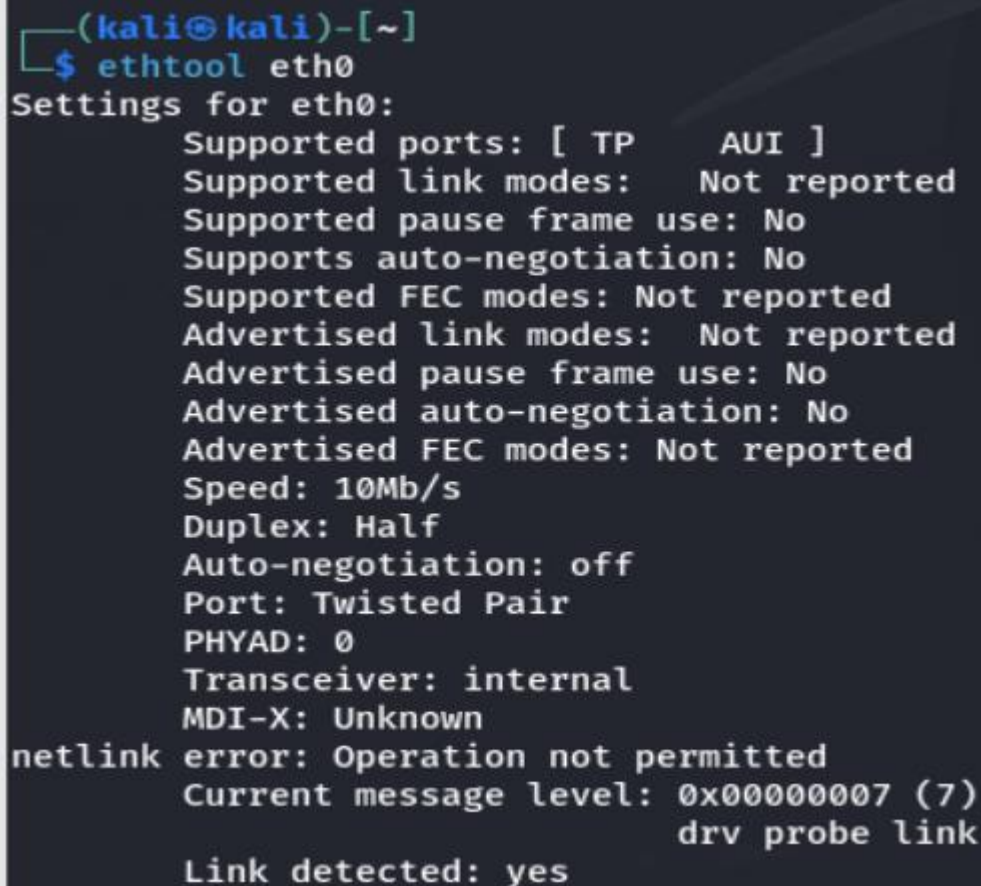
```
ethtool eth0
```

Expected Output:

```
Settings for eth0:  
Speed: 1000Mb/s  
Duplex: Full  
Auto-negotiation: on  
Link detected: yes
```

What to Check:

- Does Speed match the expected NIC/switch port speed?
- Is Duplex set to Full (Half-duplex often indicates a mismatch)?
- Is Link detected: yes?

 Screenshot:

```
(kali@kali)-[~]  
$ ethtool eth0  
Settings for eth0:  
Supported ports: [ TP      AUI ]  
Supported link modes:      Not reported  
Supported pause frame use: No  
Supports auto-negotiation: No  
Supported FEC modes: Not reported  
Advertised link modes:      Not reported  
Advertised pause frame use: No  
Advertised auto-negotiation: No  
Advertised FEC modes: Not reported  
Speed: 10Mb/s  
Duplex: Half  
Auto-negotiation: off  
Port: Twisted Pair  
PHYAD: 0  
Transceiver: internal  
MDI-X: Unknown  
netlink error: Operation not permitted  
Current message level: 0x00000007 (7)  
                        drv probe link  
Link detected: yes
```

 SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: ethtool

24. tcpdump

Purpose:

Captures live network packets on an interface for deep packet-level inspection.

Syntax:

```
tcpdump -i eth0
tcpdump -i eth0 port 80
tcpdump -i eth0 host 192.168.1.10
```

NOC Use Case:

A NOC Analyst uses tcpdump for advanced troubleshooting when higher-level tools aren't conclusive — for example, confirming whether SYN packets are actually leaving the host or being silently dropped.

Example:

```
tcpdump -i eth0 port 80 -c 5
```

Expected Output:

```
10:00:01.123456 IP 192.168.1.100.52344 > 93.184.216.34.80: Flags [S], seq 123456
10:00:01.135678 IP 93.184.216.34.80 > 192.168.1.100.52344: Flags [S.], seq 654321
10:00:01.135900 IP 192.168.1.100.52344 > 93.184.216.34.80: Flags [.], ack 654322
```

What to Check:

- Are SYN packets leaving and SYN-ACK packets returning (successful handshake)?
- Are packets retransmitted repeatedly (sign of loss)?
- Is traffic reaching the expected interface at all?

**Screenshot:**

```
(kali@kali)~$ sudo tcpdump -i eth0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
00:44:24.737767 ARP, Request who-has 192.168.149.254 tell kali, length 28
00:44:24.738286 ARP, Reply 192.168.149.254 is-at 00:50:56:f5:1e:cd (oui Unknown), length 46
00:44:24.738296 IP kali.bootpc > 192.168.149.254.bootps: BOOTP/DHCP, Request from 00:0c:29:af:f3:58 (oui Unknown), length 282
00:44:24.739704 IP 192.168.149.254.bootps > kali.bootpc: BOOTP/DHCP, Reply, length 300
00:44:24.848331 IP kali.mdns > 224.0.0.251.mdns: 0 PTR (QM)? 254.149.168.192.in-addr.arpa. (46)
00:44:25.078758 IP kali.41746 > 192.168.149.254.5355: Flags [S], seq 1668702059, win 32120, options [mss 1460,sackOK,TS val 890283385 ecr 0,nop,wscale 7,tfo
cookie:req,nop,nop], length 0
00:44:26.095246 IP kali.mdns > 224.0.0.251.mdns: 0 PTR (QM)? 254.149.168.192.in-addr.arpa. (46)
00:44:28.330372 IP kali.mdns > 224.0.0.251.mdns: 0 PTR (QM)? 254.149.168.192.in-addr.arpa. (46)
00:44:32.605405 IP kali.mdns > 224.0.0.251.mdns: 0 PTR (QM)? 251.0.0.224.in-addr.arpa. (42)
00:44:33.831877 IP kali.mdns > 224.0.0.251.mdns: 0 PTR (QM)? 251.0.0.224.in-addr.arpa. (42)
00:44:36.082969 IP kali.mdns > 224.0.0.251.mdns: 0 PTR (QM)? 251.0.0.224.in-addr.arpa. (42)
^C
11 packets captured
11 packets received by filter
0 packets dropped by kernel
```

**SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: tcpdump

25. route

Purpose:

The legacy command for displaying (and modifying) the kernel IP routing table, similar to ip route.

Syntax:

```
route -n
```

NOC Use Case:

A NOC Analyst uses `route -n` on older systems or in scripts that still expect the legacy net-tools output format for the routing table.

Example:

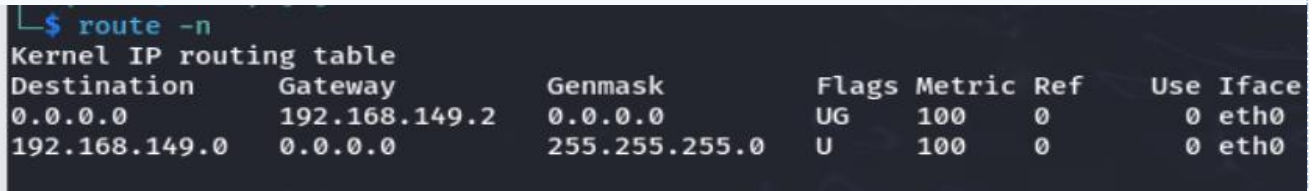
```
route -n
```

Expected Output:

```
Kernel IP routing table
Destination Gateway      Genmask      Flags  Iface
0.0.0.0     192.168.1.1    0.0.0.0      UG     eth0
192.168.1.0 0.0.0.0        255.255.255.0 U      eth0
```

What to Check:

- Is there a default route (0.0.0.0) with the correct gateway?
- Is the Iface correct for that route?

 Screenshot:

```
$ route -n
Kernel IP routing table
Destination Gateway      Genmask      Flags Metric Ref  Use Iface
0.0.0.0     192.168.149.2 0.0.0.0      UG    100    0    0 eth0
192.168.149.0 0.0.0.0        255.255.255.0 U     100    0    0 eth0
```

 SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: route

Section 6 System & NOC Troubleshooting

These commands round out the NOC toolkit by helping an analyst check service health, review system logs, and confirm the host itself has the resources to operate normally.

26. systemctl

Purpose:

Manages and reports on the status of systemd services (start, stop, restart, enable, and check status).

Syntax:

```
systemctl status nginx
systemctl restart networking
systemctl list-units --failed
```

NOC Use Case:

A NOC Analyst uses systemctl to confirm a network-related service (SSH, networking, DNS resolver) is active and running, and to restart it if it has failed.

Example:

```
systemctl status nginx
```

Expected Output:

- ssh.service - OpenBSD Secure Shell server
Loaded: loaded (/lib/systemd/system/ssh.service; enabled)
Active: active (running) since Sun 2026-07-26 09:50:12 UTC; 10min ago
Main PID: 812 (sshd)

What to Check:

- Is the service Active: active (running)?
- Is it enabled (will start on boot)?
- Are there recent restart/failure timestamps?



Screenshot:

```

(kali㉿kali)-[~]
$ systemctl restart networking

(kali㉿kali)-[~]
$ systemctl list-units --failed
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
● apache2.service                  loaded failed failed The Apache HTTP Server
● fwupd-refresh.service            loaded failed failed Refresh fwupd metadata and update motd

Legend: LOAD    → Reflects whether the unit definition was properly loaded.
          ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
          SUB     → The low-level unit activation state, values depend on unit type.

2 loaded units listed.

(kali㉿kali)-[~]
$ sudo systemctl status nginx # or apache2
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-07-27 00:14:12 CDT; 35min ago
     Invocation: 87502c8e24724f358c1cf547747c925c
       Docs: man:nginx(8)
    Main PID: 4354 (nginx)
      Tasks: 5 (limit: 19099)
     Memory: 5.1M (peak: 5.7M)
        CPU: 37ms
    CGroup: /system.slice/nginx.service
            └─4354 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
              └─4355 "nginx: worker process"
                └─4356 "nginx: worker process"
                  └─4357 "nginx: worker process"
                    └─4358 "nginx: worker process"

Jul 27 00:14:12 kali systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Jul 27 00:14:12 kali systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.

```



SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: systemctl

27. journalctl

Purpose:

Queries and displays logs collected by systemd's journal, including kernel, service, and boot logs.

Syntax:

```

journalctl -u ssh
journalctl -xe
journalctl --since "10 minutes ago"

```

NOC Use Case:

A NOC Analyst uses journalctl to review recent log entries for a failing service, correlating the exact time of a network incident with error messages in the logs.

Example:

```
journalctl -u ssh --since "10 min ago"
```

Expected Output:

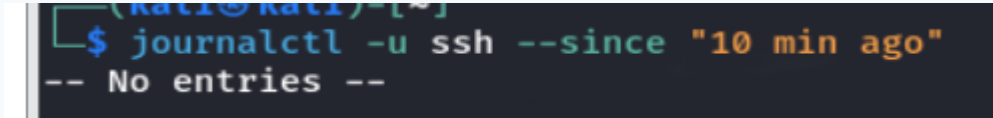
```

Jul 26 09:50:12 noc-server01 sshd[812]: Server listening on 0.0.0.0 port 22.
Jul 26 09:52:03 noc-server01 sshd[915]: Accepted password for admin from
192.168.1.55

```

What to Check:

- Are there error or warning entries around the time of the incident?
- Does the log confirm the service started/restarted correctly?

🖼️ Screenshot:**🖼️ SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: journalctl

28. ps**Purpose:**

Lists currently running processes, their PIDs, CPU/memory usage, and command line.

Syntax:

```
ps aux
ps aux | grep nginx
```

NOC Use Case:

A NOC Analyst uses `ps` to confirm a network service's process is actually running, check its PID for use with other tools like `lsof`, and spot runaway processes consuming resources.

Example:

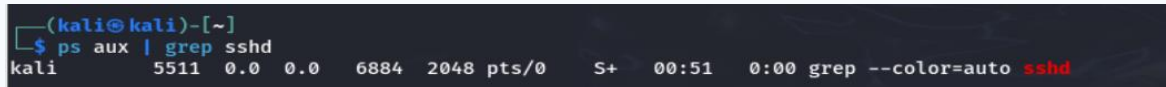
```
ps aux | grep sshd
```

Expected Output:

```
root  812  0.0  0.3 16544  6100 ?  Ss  09:50   0:00 /usr/sbin/sshd -D
```

What to Check:

- Is the expected process actually running?
- Is CPU/MEM usage abnormally high?
- Does the PID match what other tools (`lsof`, `ss`) report?

🖼️ Screenshot:**🖼️ SCREENSHOT PLACEHOLDER**

Insert your terminal screenshot here showing the output of: ps

29. top

Purpose:

Provides a live, continuously updating view of system resource usage — CPU, memory, and running processes.

Syntax:

```
top
```

NOC Use Case:

A NOC Analyst uses top to check whether high CPU or memory usage on a server is the underlying cause of degraded network performance or dropped connections.

Example:

```
top
```

Expected Output:

```
top - 10:05:12 up 2:15,  1 user,  load average: 0.15, 0.20, 0.18
Tasks: 110 total,   1 running, 109 sleeping
%Cpu(s):  3.2 us,  1.1 sy,   0.0 ni, 95.4 id
MiB Mem :  3900.0 total,  2100.0 free,   900.0 used
```

What to Check:

- Is load average unusually high for the number of CPU cores?
- Is any single process consuming excessive CPU or memory?
- Is free memory critically low?



Screenshot:

```

top - 00:51:56 up 1:27, 2 users, load average: 0.11, 0.09, 0.09
Tasks: 237 total, 1 running, 236 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.1 us, 0.9 sy, 0.0 ni, 99.7 id, 0.0 wa, 0.0 hi, 0.3 si, 0.0 st
MiB Mem : 16000.2 total, 14084.8 free, 1172.5 used, 1057.6 buff/cache
MiB Swap: 975.0 total, 975.0 free, 0.0 used, 14827.7 avail Mem

  PID USER      PR  NI   VIRT    RES    SHR  S  %CPU  %MEM    TIME+  COMMAND
 1804 kali      20   0 5048124 419956 143488 S   2.0   2.6   1:22.73 gnome-shell
 1606 kali      20   0 366440 95412 63944 S   1.2   0.6   0:24.89 Xorg
 2746 kali      20   0 626816 51740 40540 S   1.0   0.3   0:13.72 gnome-terminal-
 1933 kali      20   0 386756 11704 6784 S   0.5   0.1   0:05.10 ibus-daemon
 5527 kali      20   0 9628 4864 2816 R   0.5   0.0   0:00.02 top
 589 root       20   0 317836 9088 7680 S   0.2   0.1   0:10.26 vmtotlsd
 1967 kali      20   0 218068 42724 29612 S   0.2   0.3   0:11.18 vmtotlsd
   1 root       20   0 23104 14452 10080 S   0.0   0.1   0:24.03 systemd
   2 root       20   0 0 0 0 S   0.0   0.0   0:00.02 kthreadd
   3 root       20   0 0 0 0 S   0.0   0.0   0:00.00 pool_workqueue_release
   4 root       0 -20 0 0 0 I   0.0   0.0   0:00.00 kworker/R-rcu_g
   5 root       0 -20 0 0 0 I   0.0   0.0   0:00.00 kworker/R-rcu_p
   6 root       0 -20 0 0 0 I   0.0   0.0   0:00.00 kworker/R-slab_
   7 root       0 -20 0 0 0 I   0.0   0.0   0:00.00 kworker/R-netns
  11 root       20   0 0 0 0 I   0.0   0.0   0:00.09 kworker/u8:0-ext4-rsv-conversion
  12 root       0 -20 0 0 0 I   0.0   0.0   0:00.00 kworker/R-mm_pe
  13 root       20   0 0 0 0 I   0.0   0.0   0:00.00 rcu_tasks_kthread
  14 root       20   0 0 0 0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
  15 root       20   0 0 0 0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
  16 root       20   0 0 0 0 S   0.0   0.0   0:00.07 ksoftirqd/0
  17 root       20   0 0 0 0 I   0.0   0.0   0:00.62 rcu_preempt
  18 root       rt  0 0 0 0 S   0.0   0.0   0:00.05 migration/0
  19 root      -51  0 0 0 0 S   0.0   0.0   0:00.00 idle_inject/0
  20 root       20   0 0 0 0 S   0.0   0.0   0:00.00 cpuhp/0
  21 root       20   0 0 0 0 S   0.0   0.0   0:00.00 cpuhp/1
  22 root      -51  0 0 0 0 S   0.0   0.0   0:00.00 idle_inject/1
  23 root       rt  0 0 0 0 S   0.0   0.0   0:00.41 migration/1
  24 root       20   0 0 0 0 S   0.0   0.0   0:00.46 ksoftirqd/1
  26 root       0 -20 0 0 0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
  27 root       20   0 0 0 0 S   0.0   0.0   0:00.00 cpuhp/2
  28 root      -51  0 0 0 0 S   0.0   0.0   0:00.00 idle_inject/2
  29 root       rt  0 0 0 0 S   0.0   0.0   0:00.41 migration/2
  30 root       20   0 0 0 0 S   0.0   0.0   0:00.06 ksoftirqd/2

```



SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: `top`

30. df

Purpose:

Displays disk space usage for all mounted filesystems.

Syntax:

```
df -h
```

NOC Use Case:

A NOC Analyst checks disk space when a service fails unexpectedly, since a full disk can prevent logging, cause services to crash, or fill up queues — all of which can look like a 'network' issue at first.

Example:

```
df -h
```

Expected Output:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	20G	12G	6.9G	64%	/
tmpfs	2.0G	0	2.0G	0%	/dev/shm

What to Check:

- Is any filesystem at or near 100% used?
- Is the root partition (/) healthy?
- Is there enough free space for logs and temp files?

 Screenshot:

```
(kali@kali)-[~]
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
udev            7.8G   0 7.8G   0% /dev
tmpfs           1.6G  1.4M  1.6G   1% /run
/dev/sda1       78G   19G  55G  26% /
tmpfs           7.9G   0 7.9G   0% /dev/shm
tmpfs           5.0M   0 5.0M   0% /run/lock
tmpfs           1.0M   0 1.0M   0% /run/credentials/systemd-journald.service
tmpfs           1.0M   0 1.0M   0% /run/credentials/systemd-tmpfiles-setup-dev-early.service
tmpfs           1.0M   0 1.0M   0% /run/credentials/systemd-udev-load-credentials.service
tmpfs           1.0M   0 1.0M   0% /run/credentials/systemd-sysctl.service
tmpfs           1.0M   0 1.0M   0% /run/credentials/systemd-tmpfiles-setup-dev.service
tmpfs           7.9G  12K  7.9G   1% /tmp
tmpfs           1.0M   0 1.0M   0% /run/credentials/systemd-tmpfiles-setup.service
tmpfs           1.6G 104K  1.6G   1% /run/user/1000
tmpfs           1.0M   0 1.0M   0% /run/credentials/systemd-resolved.service
```

 SCREENSHOT PLACEHOLDER

Insert your terminal screenshot here showing the output of: df

✓ Conclusion

This reference brings together 30 of the most used Linux commands for network configuration, connectivity troubleshooting, DNS diagnostics, port and connection analysis, and system health monitoring. Together, they form a practical, end-to-end troubleshooting workflow that mirrors the day-to-day responsibilities of a NOC Analyst: verify configuration, test reachability, resolve DNS, confirm ports and services, and check the underlying system health when something looks like a network problem but isn't.

Building this document hands-on running each command in a live Ubuntu environment and capturing real terminal output demonstrates practical Linux networking competency that goes beyond memorizing command syntax. This project is intended to be shared on GitHub and referenced in interviews as evidence of applied, job-ready NOC troubleshooting skills.